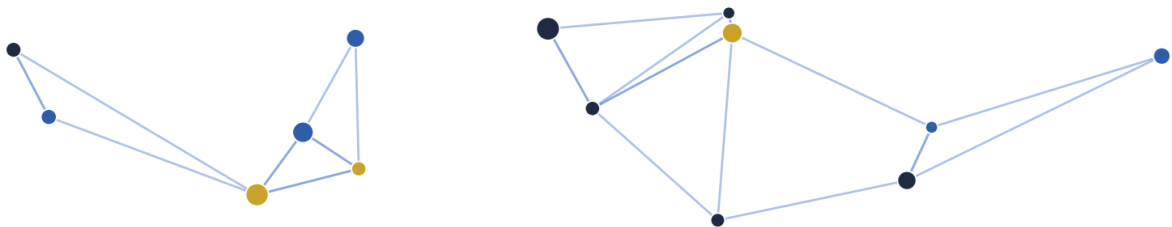


PROJECT DOCUMENTATION

Astra AI

AI-Powered Web Chat Application

A Flask-based web application integrating AI chat and image-generation models through the Experiential Labs API gateway



Project Type	AI-powered web application
Backend	Python (Flask)
Frontend	HTML, CSS, JavaScript
AI Gateway	Experiential Labs (OpenAI-compatible API)
Development Environment	Kali Linux
Status	Core chat functionality complete — image generation in progress

CONTENTS

Table of Contents

- | | |
|---------------------------------------|-------------------------------------|
| 1. Project Overview | 15. First Chat API Test |
| 2. Project Architecture | 16. Backend Error Handling |
| 3. Development Environment | 17. Image Generation Investigation |
| 4. Project Files | 18. First Image Generation Test |
| 5. Creating the Flask Backend | 19. Testing the Responses API |
| 6. Connecting the AI API | 20. Lesson From Image Testing |
| 7. API Key Security | 21. Current Image Generation Status |
| 8. Testing API Authentication | 22. Security Considerations |
| 9. Discovering Available Models | 23. Current Architecture |
| 10. Selecting the AI Model | 24. Current Project Status |
| 11. Free Model Investigation | 25. Future Improvements |
| 12. Frontend Development | 26. Key Technical Lessons Learned |
| 13. Frontend-to-Backend Communication | 27. Conclusion |
| 14. Running the Application | |
-
-

Project Overview

1 Introduction

Astra AI is a full-stack AI-powered web application developed to provide users with an interactive conversational AI experience through a web interface.

The project combines a frontend web interface, a Python Flask backend, and an external AI API. The backend is responsible for communicating securely with the AI service, while the frontend provides the user interface through which users can send messages and receive AI-generated responses.

The project was developed as a practical learning project to understand how modern AI applications communicate with external AI APIs and how frontend, backend, authentication, model selection, and error handling work together in a real application.

2 Project Objectives

The main objectives of the Astra AI project were:

- Build a functional AI chat website.
 - Develop a frontend similar to modern AI chat applications.
 - Create a Flask-based backend to handle AI requests.
 - Connect the application to the Experiential Labs AI API.
 - Secure the API key using an environment variable.
 - Discover and test available AI models.
 - Understand how AI model access works through an API gateway.
 - Implement frontend-to-backend communication.
 - Handle API errors correctly.
 - Investigate AI image-generation capabilities.
 - Understand the difference between model availability and actual model access.
 - Understand how different AI models may require different API capabilities or serving paths.
-

3 Application Name

The project is named:

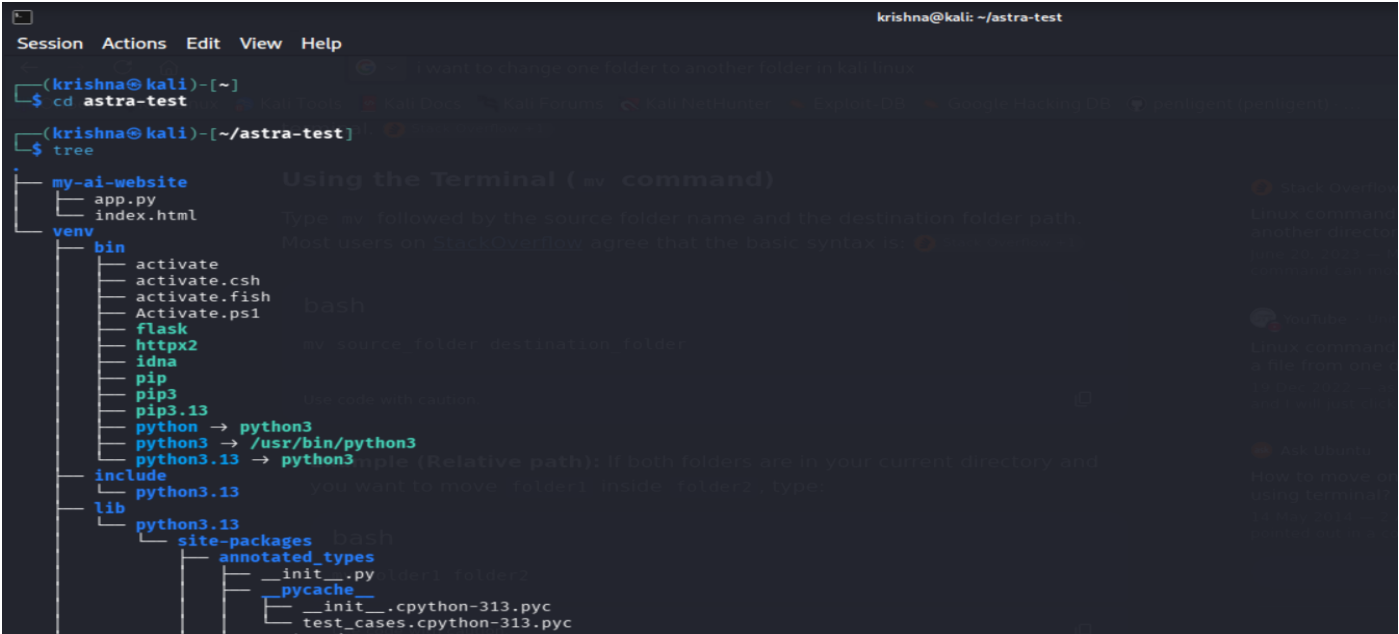
Astra AI — AI-Powered Web Application

The name Astra AI was selected as the identity of the AI assistant used by the website.

The application is designed to provide a clean conversational interface where users can enter questions or instructions and receive responses from an AI model.

4 Main Technologies Used

Technology	Purpose
HTML	Structure of the frontend
CSS	User interface design and responsive layout
JavaScript	Frontend interaction and API communication
Python	Backend programming
Flask	Web application backend
Experiential Labs API	AI model access
REST API	Communication with the AI service
Environment Variables	Secure API-key storage
Kali Linux	Development environment



5 Application Architecture

Astra AI follows a simple full-stack architecture consisting of three primary layers:

- **Frontend Layer** — HTML, CSS, and JavaScript provide the user interface.
- **Backend Layer** — Flask processes requests and manages communication with the AI service.
- **AI Service Layer** — Experiential Labs provides access to the selected AI model.

Architecture Flow

1. **User** enters a message in the Astra AI web interface.

2. **JavaScript** sends the message to the Flask /chat endpoint.
3. **Flask Backend** validates and processes the request.
4. **Backend** authenticates with the Experiential Labs API using the protected API key.
5. **AI Model** processes the user's request.
6. **API Response** is received by the Flask backend.
7. **Flask** returns the response to the frontend.
8. **Frontend** displays the AI response to the user.

This architecture keeps the API key on the backend and prevents it from being directly exposed to users through the browser.

6 Development Environment

The application was developed and tested in a **Kali Linux** environment.

The project used Python and a Python virtual environment for backend development. Flask was used to create the local web server and API endpoint.

The frontend was developed using:

- HTML
- CSS
- JavaScript

The application was tested locally using the Flask development server.

7 Project Directory

The main project directory used during development was:

~/my-ai-website

A Python virtual environment was also used:

~/astra-test/venv

Using a virtual environment helped keep the Python dependencies isolated from the rest of the operating system.

8 Initial Project Files

The initial application consisted primarily of:

my-ai-website/

├─ app.py

└─ index.html

app.py

This file contains the Flask backend.

Its responsibilities include:

- Starting the Flask application.
- Receiving chat requests from the frontend.
- Communicating with the AI API.
- Handling API responses.
- Returning results to the frontend.
- Handling errors generated during API communication.

index.html

This file contains the main frontend interface.

</> Html Code:

```
(venv)krishna@kali: ~/astra-test/my-ai-website | krishna@kali: ~/astra-test/my-ai-website | index.html
GNU nano 8.7.1
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Astra - AI Workspace</title>
<style>
:root {
  --bg: #0a0b10;
  --panel: #11131b;
  --panel2: #171a24;
  --border: rgba(255,255,255,.08);
  --text: #f5f7fb;
  --muted: #8b93a7;
  --accent: #8b5cf6;
  --accent2: #6366f1;
  --user: #252a3a;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  background: var(--bg);
  color: var(--text);
  font-family: Inter, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  height: 100vh;
  overflow: hidden;
}
button { font-family: inherit; }
button:focus-visible, textarea:focus-visible, input:focus-visible {
  outline: 2px solid rgba(139,92,246,.65);
}
```

It provides the structure for:

- Astra AI branding.
- Sidebar navigation.
- Welcome screen.
- Chat messages.
- Suggestion cards.
- Message input.
- Send functionality.
- Responsive interface elements.

9 Flask Backend

Flask was selected as the backend framework because it is lightweight and suitable for building small web applications and APIs.

The backend exposes a chat endpoint that the frontend can communicate with.

The basic endpoint used by the application is:

POST /chat

The frontend sends the user's message to this endpoint.

The Flask backend then processes the request and communicates with the external AI service.

This design keeps the API credentials on the backend instead of exposing them to the browser.

Flask was used to create the backend server. The backend receives the user's message from the frontend and sends the request to the AI API.

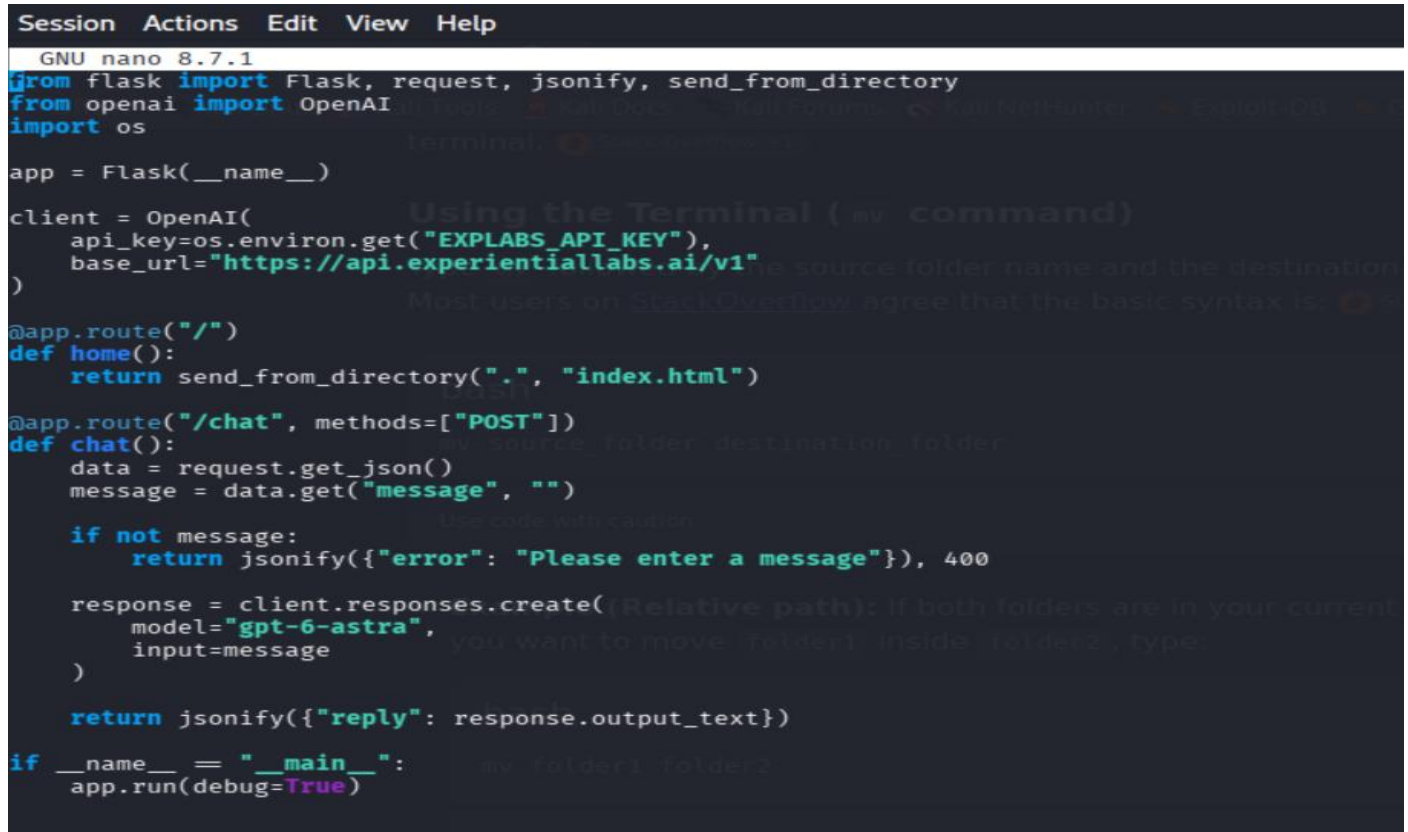
Then immediately below it:

Source Code:

app.py

your actual Flask backend code here

</> Python Code:



```
Session Actions Edit View Help
GNU nano 8.7.1
from flask import Flask, request, jsonify, send_from_directory
from openai import OpenAI
import os

app = Flask(__name__)

client = OpenAI(
    api_key=os.environ.get("EXPLABS_API_KEY"),
    base_url="https://api.experientiallabs.ai/v1"
)

@app.route("/")
def home():
    return send_from_directory(".", "index.html")

@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json()
    message = data.get("message", "")

    if not message:
        return jsonify({"error": "Please enter a message"}), 400

    response = client.responses.create(
        model="gpt-6-astra",
        input=message
    )

    return jsonify({"reply": response.output_text})

if __name__ == "__main__":
    app.run(debug=True)
```

10 AI API Integration

The project uses the **Experiential Labs API** to access AI models.

The API gateway used during development was:

`https://api.experientiallabs.ai/v1`

The API uses bearer-token authentication:

Authorization: Bearer <API_KEY>

The API key was stored in an environment variable instead of being written directly into the frontend source code.

The environment variable used was:

`EXPLABS_API_KEY`

This approach reduces the risk of accidentally exposing the API key through the website's client-side code.

11 Model Discovery

Before selecting a model for the application, available models were investigated through the API.

The model endpoint used for testing was:

`GET /v1/models`

This was useful because the public model catalog and the models actually accessible through an API key are not necessarily the same.

During the investigation, several models were identified, including:

- `gpt-5-mini`
- `gpt-5-image`
- `gpt-5-image-mini`
- `gemini-2.5-flash-image`
- `gemini-3-pro-image`
- `gemini-3.1-flash-image`
- `gpt-5.6-luna`
- `gpt-6-astra`

The investigation demonstrated that simply seeing a model in a catalog does not automatically mean that the account can successfully use that model.

12 First Model Access Investigation

The first major model-access investigation involved the `gpt-6-astra` model.

Although the model appeared to be available in the model catalog, attempting to use it through the application resulted in an HTTP **429** response.

The investigation showed that access to certain free or promotional models required an additional account verification step involving a \$1 charge.

At that stage, the verification was not completed.

This became an important lesson in understanding the difference between:

- A model being listed in the catalog.
- A model being available to an API key.
- A model being available under the current account's pricing or promotional conditions.

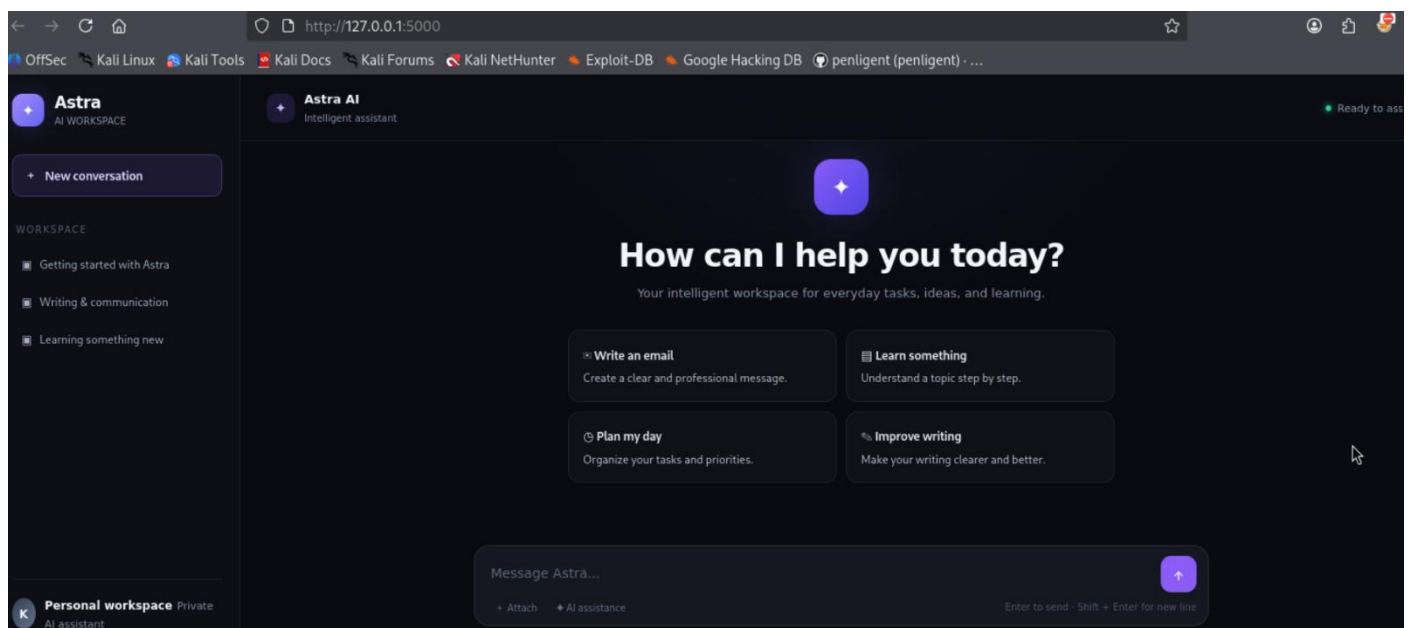
13 Frontend Development

The Astra AI frontend was designed to provide a modern AI-chat experience.

The interface includes:

- Astra AI branding.
- Sidebar navigation.
- Model information.
- Welcome screen.
- Suggestion cards.
- User messages.
- AI responses.
- Message input area.
- Send functionality.
- Responsive styling for different screen sizes.

The interface was designed using HTML and CSS, while JavaScript was used to provide dynamic interaction.



14 Frontend-to-Backend Communication

The frontend communicates with the Flask backend using JavaScript.

When the user enters a message, the browser sends a POST request to:

/chat

The request contains the user's message in JSON format.

A simplified example is:

```
{  
  "message": "Hello Astra"  
}
```

The Flask backend receives the request and processes it.

The backend then communicates with the AI API and returns the result to the browser.

Source Code:

</> JavaScript

```
const response = await fetch("/chat", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json"  
  },  
  body: JSON.stringify({  
    message: message  
  })  
});
```

15 Running the Application

The Flask application was run locally during development.

The local application was accessible through:

http://127.0.0.1:5000

This allowed the frontend and backend to be tested together before considering any production deployment.

```
Session Actions Edit View Help
(venv)-(krishna@kali)-[~/astra-test/my-ai-website]
$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 916-282-603
```

16 First Chat API Test

The first complete chat test was performed through the Astra AI website.

The frontend successfully sent the user's request to the Flask backend.

However, the backend was configured to use gpt-6-astra, and the external API returned an HTTP **429** error.

As a result, the browser received an HTTP **500** response from the Flask backend.

This demonstrated an important difference between an external API error and the application's own error response.

The external service returned:

HTTP 429

while the Flask application returned:

HTTP 500

because the backend did not initially handle the upstream API failure correctly.

17 Backend Error Handling

Error handling was then added to the Flask backend.

The backend was designed to:

- Validate incoming requests.
- Catch exceptions.
- Record useful error information.
- Prevent uncontrolled application crashes.
- Return a controlled JSON response to the frontend.

A simplified structure is:

try:

```
# Process the AI request
```

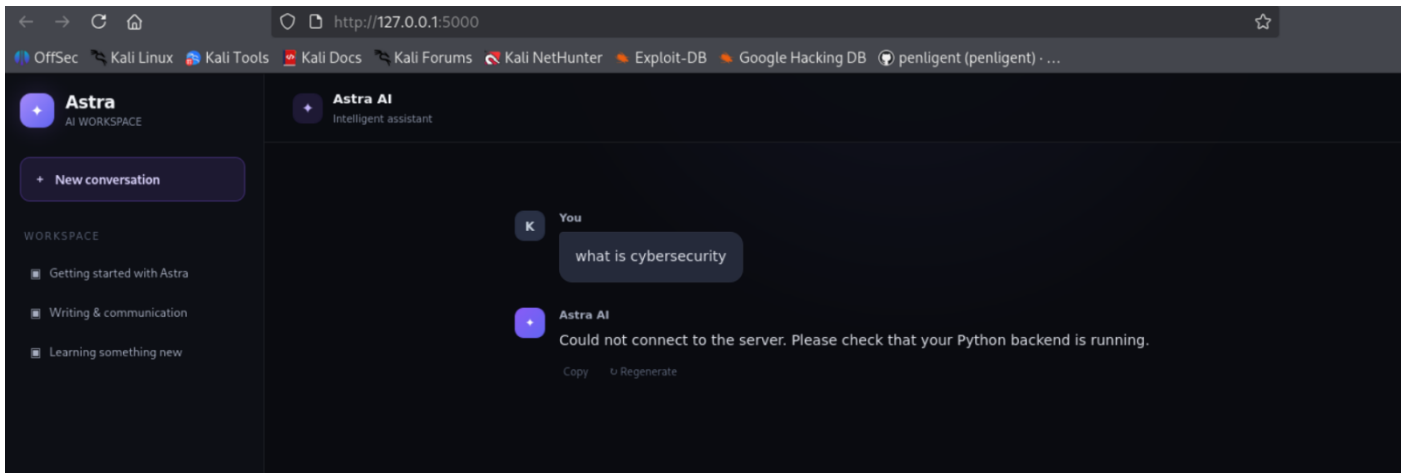
pass

except Exception as e:

```
# Handle the error
```

```
print(e)
```

The purpose was not simply to hide errors, but to make the application's behavior more predictable when the external AI service fails.



18 Image Generation Investigation

After establishing the basic chat functionality, image generation became an important part of the project.

The objective was to allow Astra AI to generate an actual image when the user enters a request such as:

Generate a Lord Krishna image

The goal was specifically to produce image output rather than simply asking the language model to describe an image or create an image-generation prompt.

The gpt-5-image model was therefore investigated.

19 Image Generation Endpoint Test

The first image-generation test attempted to use:

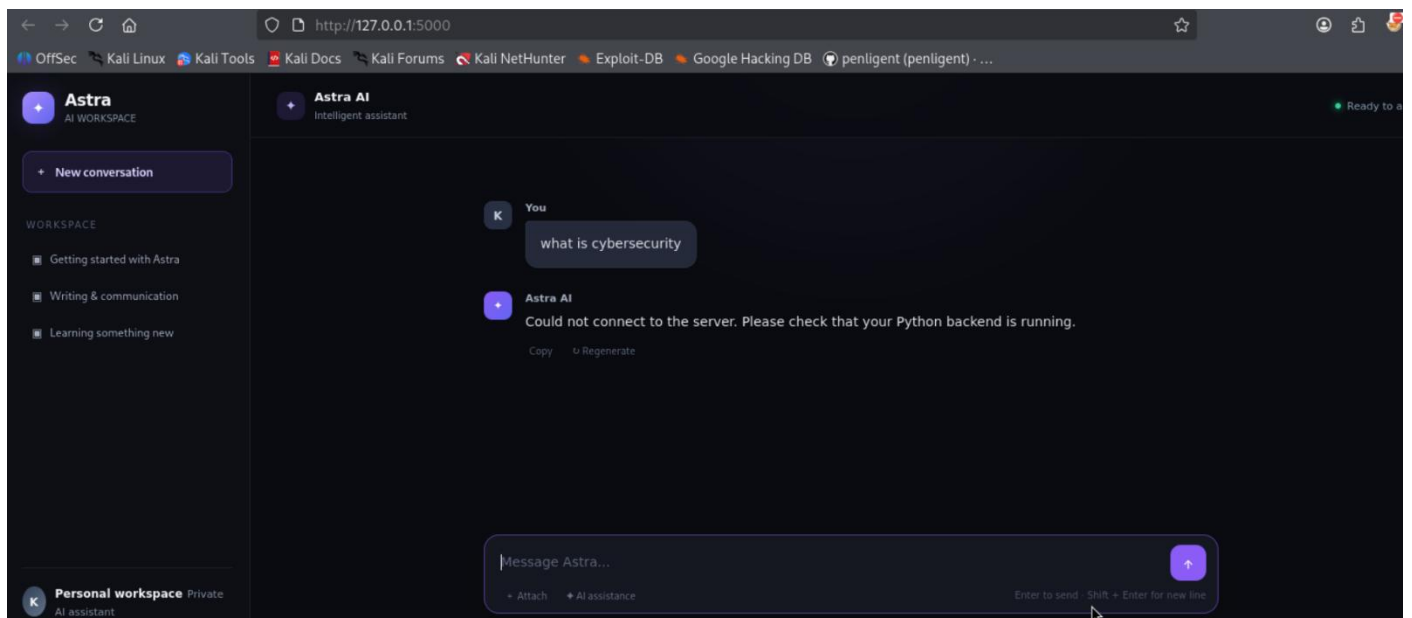
```
/v1/images/generations
```

The API returned:

```
Unsupported serving path: /v1/images/generations
```

This showed that the Experiential Labs gateway did not support the assumed image-generation endpoint for this model.

Therefore, the project could not simply copy an image-generation implementation from another API provider and expect it to work.



20 Responses API Investigation

The next test used the Responses API with the gpt-5-image model.

The request was accepted by the API.

The response contained:

status: completed

and:

error: null

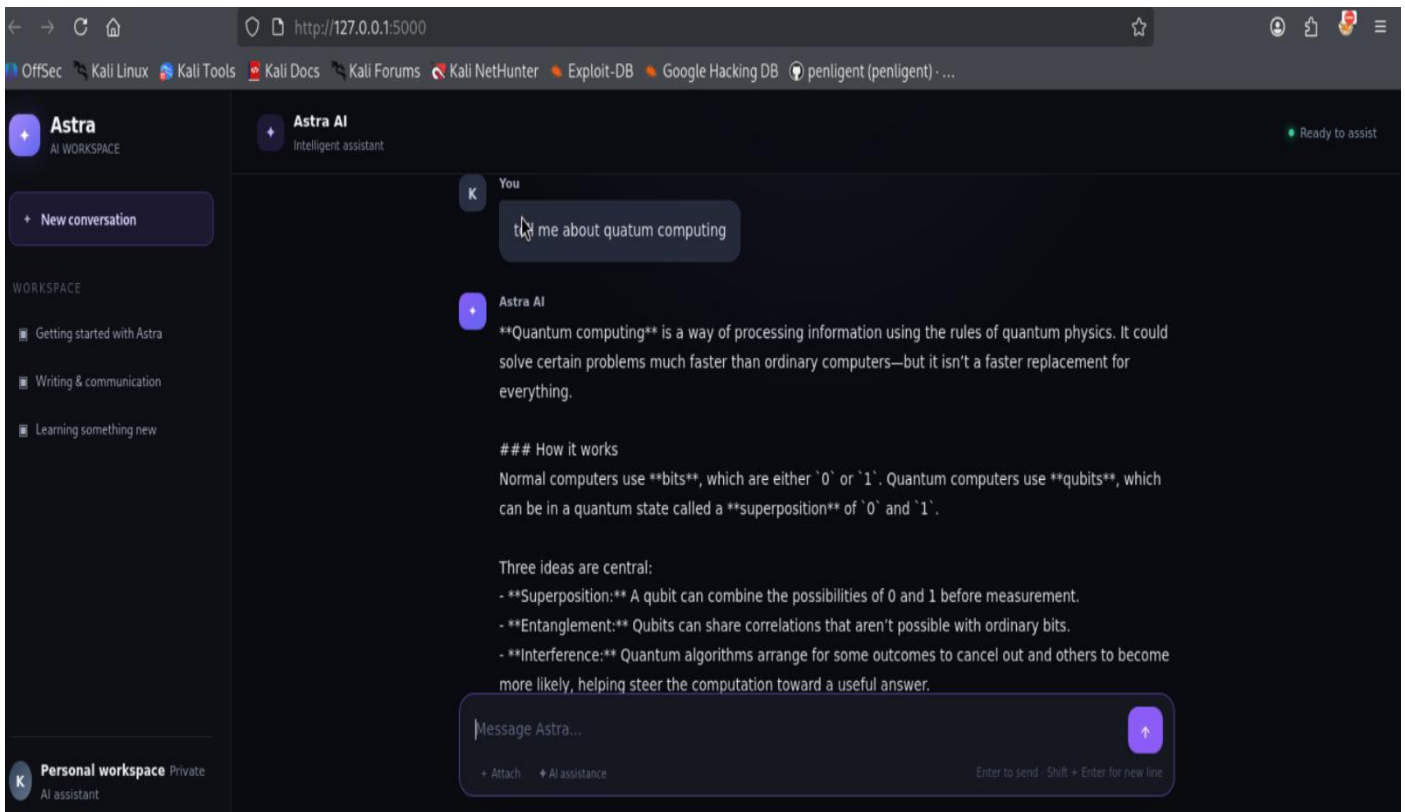
However, the returned response contained:

output: []

No actual image data was returned to the application.

The request also consumed API credits.

This result demonstrated that successful API processing does not necessarily mean that the expected output was actually produced.



21 Important Lesson From Image Testing

The image-generation investigation produced several important technical lessons.

A model name alone does not tell us:

- Which API endpoint should be used.
- Which request format is required.
- Whether the model can return image data.
- Whether image output is supported through a particular gateway.
- What format the response will use.

Therefore, before implementing image generation permanently, the supported model capabilities, serving paths, request format, and response format must be verified.

This also prevents unnecessary API-credit consumption from repeated unsuccessful tests.

```
(venv)-(krishna@kali)-[~/astra-test/my-ai-website]
└─$ python -c 'import os; k=os.getenv("EXPLABS_API_KEY",""); print("length:",len(k)); print("prefix:",k[:4]); print("suffix:",k[-4:])'
length: 44
prefix: xpl_
suffix: bb1a

(venv)-(krishna@kali)-[~/astra-test/my-ai-website]
└─$ curl -s https://api.experientiallabs.ai/v1/models \
-H "Authorization: Bearer $EXPLABS_API_KEY" | head -c 200
{"object":"list","data":[{"id":"aion-2.0","object":"model","created":0,"owned_by":"exp"},{"id":"aion-3.0","object":"model","created":0,"owned_by":"exp"},{"id":"el","created":0,"owned_by":"exp"}]}
```

22 Current Image Generation Status

At this stage of the project, normal AI chat functionality and the frontend/backend foundation have been established.

Image generation has been investigated but is **not yet fully implemented**.

The main issue is determining the correct image-output mechanism supported by the Experiential Labs gateway for the selected image model.

Further implementation should only be performed after confirming the correct API documentation and response format.

23 Security Considerations

Security was considered during the development of the application.

The API key should never be placed directly inside:

index.html

or exposed through client-side JavaScript.

Instead, the API key is stored using:

EXPLABS_API_KEY

as an environment variable.

The project should also prevent sensitive files from being committed to source-control repositories.

For example:

.env

__pycache__/

venv/

can be included in .gitignore.

This helps prevent accidental exposure of API credentials and local development files.

24 Future Scope

The application may later be expanded with:

- User authentication
- Database integration
- Chat history
- Conversation management
- AI model selection

- Streaming responses
 - Image generation
 - Image analysis
 - File uploads
 - Document analysis
 - User profiles
 - Rate limiting
 - Usage tracking
 - Production deployment
 - Improved security controls
-

25 Key Technical Lessons Learned

API Authentication

API keys must be protected and should not be exposed in frontend code.

API Model Discovery

Available models should be checked through the API instead of assuming that every catalog model is accessible.

HTTP Error Investigation

An HTTP error such as 429 should be investigated rather than treated as a generic application failure.

Model Availability vs. Access

A model can exist in the provider's catalog while still being unavailable to a particular account or requiring additional verification.

API Endpoint Compatibility

Different models may require different serving paths or request formats.

Response Validation

A successful API request does not necessarily mean the expected content was returned. The actual response must be inspected.

Backend Error Handling

External API failures should be handled properly by the Flask backend so that the frontend receives a controlled response.

26 Conclusion

The Astra AI project established the foundation for a full-stack AI web application.

The project progressed from creating a basic Flask backend and frontend interface to connecting an external AI API, discovering available models, testing model access, handling API errors, and investigating image-generation capabilities.

The development process also demonstrated that AI API integration requires more than simply selecting a model name. Authentication, model availability, account access, serving paths, request formats, response formats, and error handling must all be considered.

The current application therefore provides a strong foundation for further development, with image generation and additional AI features forming part of the next stage of the project.